Agenda

- Docker History & Origins
- What is Docker?
- Why Use Docker?
- Docker Architecture
- Under the Hood: LXC, Namespaces, Cgroups
- Hands-On Lab-01: Getting Started

A Brief History

2010 dotCloud (PaaS) starts working on internal container tools.

2013 Docker launched as open-source by Solomon Hykes at PyCon.

2015 Docker Compose launched.

2019+ Kubernetes becomes the standard for orchestration.

Today Docker is the de facto standard for containerization.

What is Docker?

Definition

An open platform for developing, shipping, and running applications.

Core Concept

It enables developers to package applications into **containers**—standardized units that include everything needed to run: code, runtime, system tools, and libraries.

"Build once, run anywhere."

Why Use Docker?

Portability

Works consistently across dev, test, and prod environments.

Efficiency

Lightweight and uses fewer resources than Virtual Machines.

CI/CD

Simplifies pipelines and automates deployment.

Microservices

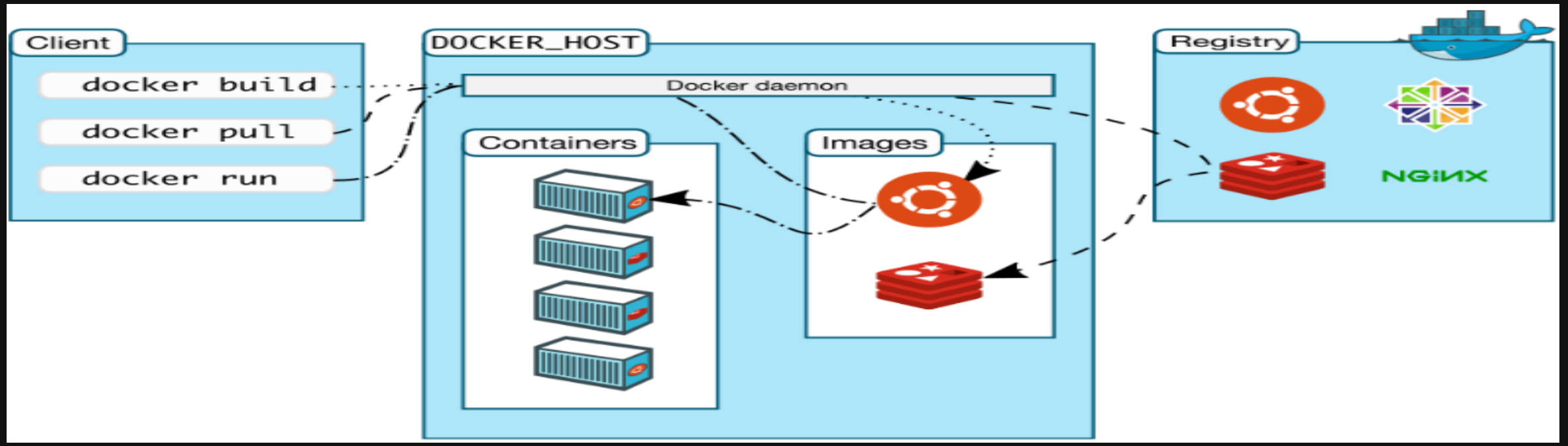
Ideal for breaking monolithic apps into smaller services.

Docker Architecture

- **Docker Daemon (Engine):** The background service running on the host that manages building, running, and distributing containers.
- **Docker Client:** The CLI tool (docker) that users interact with.
- **Docker Registry:** A store for Docker images (e.g., Docker Hub).
- **Images & Containers:** The build artifacts and the running instances.

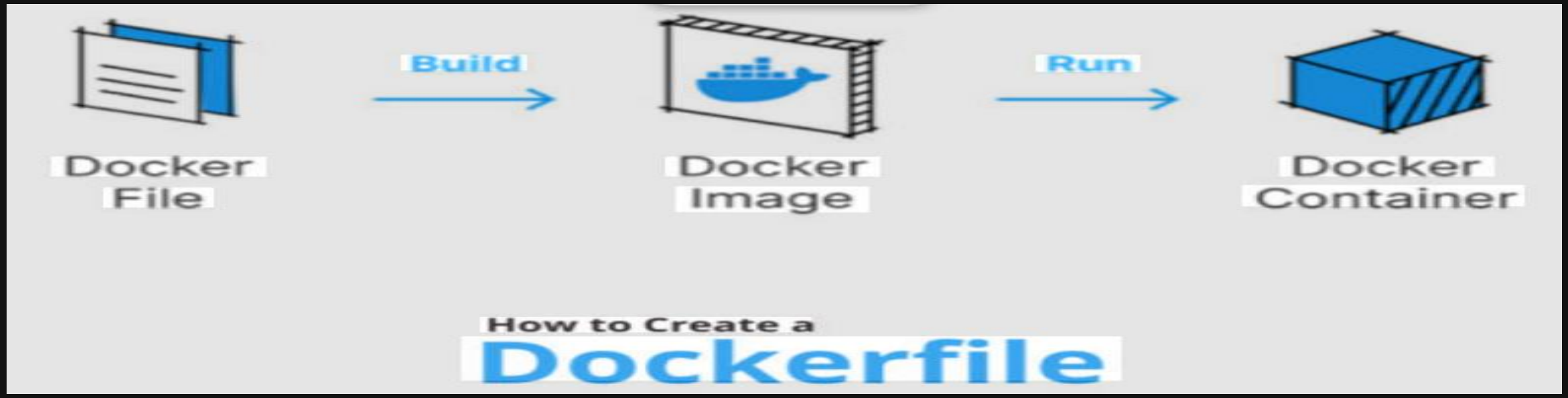
Docker Architecture

Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface. Another Docker client is Docker Compose, that lets you work with applications consisting of a set of containers



Docker Architecture

Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface. Another Docker client is Docker Compose, that lets you work with applications consisting of a set of containers

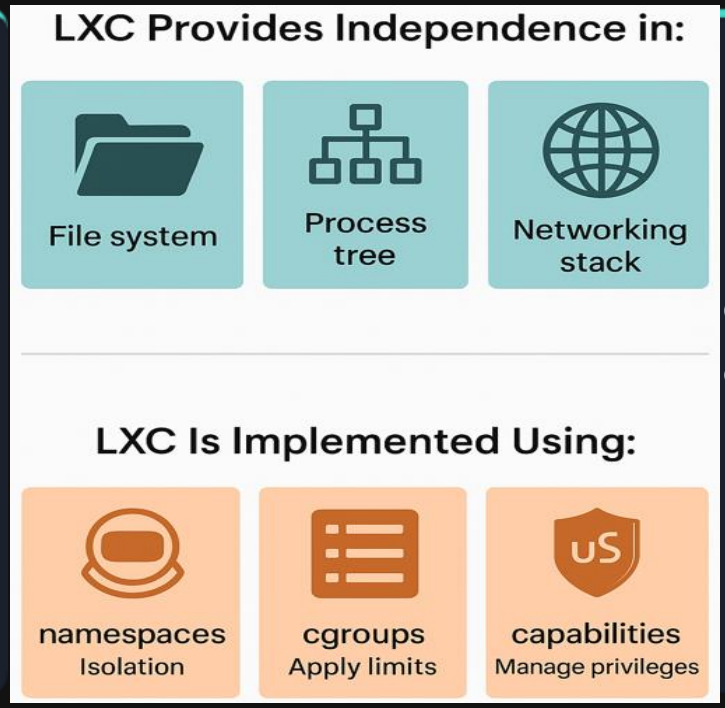


Under the Hood: LXC

Namespaces (Isolation)

Provides the isolation that makes containers feel like separate machines.

- **PID:** Process isolation
- **NET:** Network stacks
- **MNT:** Mount points
- **UTS:** Hostnames



Cgroups (Limits)

Control Groups manage and limit the resources a container can use.

- **CPU:** Core usage limits
- **RAM:** Memory limits
- **I/O:** Disk read/write limits

Hands-On – Lab 01

Getting Started

- Install Docker Desktop (Windows/Mac) or Engine (Linux).
- Verify installation.
- Run your first container.

```
# 1. Check version
docker --version
Docker version 20.10.12, build e91ed57
# 2. Run hello-world
docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
# 3. List images
docker images
REPOSITORY TAG IMAGE ID CREATED
hello-world latest feb5d9fea6a5 2 months ago
```

Questions?

Thank you for participating.

Feel free to reach out:
https://dinghy-e2e.com